

Auditing Model Substitution in LLM APIs:

Why Software Tests Fail and Hardware Attestation Wins

ML researchers • AI security practitioners • cloud infrastructure engineers

Source: arxiv.org/abs/2504.04715

Commercial LLM API providers may covertly substitute cheaper models. This work shows software-based detection is unreliable and proposes Trusted Execution Environments as a provably secure alternative.

The Model Substitution Problem: Economic Incentives Drive Covert Swaps

Commercial LLM API providers charge users for access to specific models, but the black-box nature of APIs means users have no guarantee the advertised model is actually being served. Providers face strong economic incentives to covertly substitute cheaper alternatives to reduce GPU costs while maintaining pricing.

Four Substitution Types:

Type 1

Smaller Model

e.g., Llama-3-8B served instead of Llama-3-70B

Type 2

Quantized Variant

e.g., FP8/INT8 served instead of FP16

Type 3

Fine-Tuned Version

e.g., updated checkpoint without user knowledge

Type 4

Different Family

e.g., Mistral served instead of Qwen

Formal Framework: Hypothesis Testing

H_0 (Null): The API serves the specified model with distribution $P_{\text{spec}}(y|x)$

H_1 (Alt): The API serves a substituted model with distribution $P_{\text{actual}}(y|x) \neq P_{\text{spec}}(y|x)$

Given only API samples (x, y) , can an auditor reliably reject H_0 ?

Conclusion: With FP8 quantization, P_{actual} is close enough to P_{spec} that the answer is almost always NO.

FP8 Quantization Preserves Most Accuracy — Making Substitution Hard to Detect

Model	MMLU	GSM8K	MATH	GPQA Diamond
Llama-3-8B-Instruct	62.69 ± 0.18	61.14 ± 3.47	20.65 ± 3.43	22.62 ± 0.22
Llama-3-8B-Instruct-FP8	62.43 ± 0.26	60.90 ± 4.10	14.91 ± 2.49	20.14 ± 0.24
Llama-3-70B-Instruct	78.05 ± 0.08	88.06 ± 1.44	35.69 ± 1.33	29.60 ± 0.51
Llama-3-70B-Instruct-FP8	77.88 ± 0.13	87.35 ± 1.24	35.75 ± 1.16	33.12 ± 0.30
Gemma-2-9b-it	71.86 ± 0.08	81.80 ± 1.35	33.41 ± 0.28	28.64 ± 2.97
Gemma-2-9b-it-FP8	71.92 ± 0.11	79.41 ± 1.14	32.53 ± 0.34	27.81 ± 3.22
Qwen2-72B-Instruct	82.18 ± 0.08	86.72 ± 1.00	37.39 ± 1.41	29.93 ± 2.71
Qwen2-72B-Instruct-FP8	81.98 ± 0.08	86.82 ± 0.97	37.67 ± 1.39	31.08 ± 1.96
Mistral-7B-Instruct-v0.3	59.15 ± 0.10	35.90 ± 4.54	8.94 ± 1.24	21.60 ± 0.17
Mistral-7B-Instruct-v0.3-FP8	58.77 ± 0.13	32.20 ± 4.02	7.68 ± 1.23	22.72 ± 0.19



Higher Temperature Amplifies Performance Gaps but Increases Noise

Temperature scaling experiments ($T = 0.2, 0.5, 1.0, 2.0$) show that higher temperatures degrade all models' performance across all four tasks (MMLU, GSM8K, MATH, GPQA Diamond). At temperature 2.0, the gap between original and FP8 substitute narrows and becomes noisy, complicating any statistical detection approach.

[figure_1.jpg]

Benchmark accuracy vs. temperature for five model families
across MMLU, GSM8K, MATH, and GPQA Diamond tasks

- $T = 0.2 \rightarrow$ near-deterministic output; substitution signal is strongest

- $T = 1.0 \rightarrow$ performance degrades; gap narrows between original and quantized

- $T = 2.0 \rightarrow$ high variance; gap shrinks to noise floor; statistical detection fails

- All four tasks (MMLU, GSM8K, MATH, GPQA Diamond) show similar pattern

Log Probability Fingerprinting Fails Due to Inference Nondeterminism

A promising detection approach compares token-level log probabilities between a local reference model and the API. However, production inference environments introduce significant nondeterminism that renders this approach unreliable.

[figure_5.jpg]

Log probability variation across inference frameworks and hardware
for Llama-3-8B-Instruct

Same Model, Different Outputs:

transformers 4.40 vs 4.50

Different tokenization ordering

vLLM 0.6.2 vs 0.8.2

Different kernel choices & KV cache behavior

A100 vs H100

Different numerical libraries & precision handling

Any two above combined

Multiplicative nondeterminism

An auditor cannot distinguish between a legitimate model on different infrastructure vs. an actual substitution. Log-probability fingerprinting is defeated at the production infrastructure level.

Randomized Routing at Low Rates Makes Statistical Detection Query-Prohibitive

Even if statistical tests could detect a pure substitution, providers can actively evade audits by routing only a fraction p of queries to the cheaper model.

Mixed Distribution: $P_{\text{mixed}}(y|x) = (1 - p) \cdot P_{\text{spec}}(y|x) + p \cdot P_{\text{sub}}(y|x)$

As $p \rightarrow 0$, P_{mixed} converges to P_{spec} . The Kullback-Leibler divergence $D(P_{\text{spec}} || P_{\text{mixed}}) \rightarrow 0$.

Effect of Substitution Rate p :

$p = 1.0$

Full substitution

Easiest to detect statistically

$p = 0.1$

10% mixed

Requires $\sim 10\times$ more queries for same power

$p = 0.01$

1% mixed

Requires $\sim 100\times$ more queries; near-impossible

$p \rightarrow 0$

Trace substitution

KL divergence ≈ 0 ; query budget $\rightarrow \infty$

Adversarial Countermeasure #2: Benchmark Evasion

Providers can detect known benchmark prompts (e.g., MMLU, GSM8K) via prompt fingerprinting and route them to the genuine model while serving substitutes for all regular traffic. This makes even a large query budget ineffective — the auditor's probes always get honest answers.

Software-Only Auditing Methods Are Fundamentally Unreliable

Comparison Matrix: Three Failure Modes

Method Type	Core Weakness	Practical Limitation
Text-Based Statistical Tests (e.g., MMD, classifier-based)	Need impractically large query budgets; fail against subtle or randomized substitutions	MMD divergence is tiny for FP8-substituted models. Randomized routing at $p < 0.1$ makes the mixed distribution indistinguishable from original even with thousands of queries. Query cost becomes prohibitive for most auditors.
Log-Probability Fingerprinting	Defeated by production-level inference nondeterminism across software versions & GPU hardware	Same model (Llama-3-8B) produces different token log-probabilities on transformers 4.40 vs 4.50, vLLM 0.6.2 vs 0.8.2, and A100 vs H100. Auditor cannot establish a stable reference distribution.
Adversarial Countermeasures (benchmark evasion, randomized routing)	Actively undermine any statistical or fingerprinting method; make detection query-prohibitive	Providers can fingerprint benchmark prompts and route them to the real model. Randomized routing with $p \rightarrow 0$ drives KL divergence to zero. No software method is robust against a strategically adversarial provider.

Trusted Execution Environments Provide Provable Cryptographic Model Integrity

Unlike software-based detection which is fundamentally reactive and unreliable, Trusted Execution Environments (TEEs) offer a proactive, hardware-enforced guarantee of model integrity.

How TEE Attestation Works:

Step 1	Provider loads model weights into TEE enclave
Step 2	Enclave executes measurement & signs with private key
Step 3	Remote attestation report sent to user (includes hash of loaded weights)
Step 4	User verifies report against expected model hash — cryptographically

TEE Properties vs. Software Methods:

Provable	Cryptographic proof that specific weights are loaded, not just statistical evidence
Efficient	Modest performance overhead (~5–15%); far cheaper than ZK proofs for LLMs
Robust	Not defeated by inference nondeterminism or adversarial routing — hardware-enforced

Practical Deployment Path: NVIDIA Confidential Computing on H100 GPUs

NVIDIA's confidential computing stack provides TEE support for GPU workloads. This allows providers to offer cryptographically attested LLM inference today — without the prohibitive computational overhead of ZKPs, which are estimated to be 10,000× slower than native inference for large models like Llama-3-70B.

Unlike ZKPs (computationally prohibitive for large models), TEEs are practical and deployable today.

Key Takeaways: Close the Verification Gap with Hardware Attestation

Five evidence-based conclusions from empirical analysis of model substitution threats and detection limitations:

1 Model substitution is economically motivated and practically undetectable via software.

FP8 quantization reduces costs significantly while benchmark drops fall within error bars, making substitution profitable and nearly invisible to all software-based detection methods.

2 Statistical tests require impractically large query budgets and fail against subtle or randomized substitutions.

MMD-based tests and classifier methods cannot distinguish FP8-substituted outputs from originals without thousands of queries — and randomized routing at $p < 0.1$ makes even that impossible.

3 Log-probability fingerprinting is defeated by production-level inference nondeterminism.

Different inference frameworks (transformers vs. vLLM) and hardware (A100 vs. H100) produce incompatible token-level log probabilities for the exact same model — auditor reference is never stable.

4 Adversarial countermeasures (randomized routing, benchmark evasion) further undermine all software methods.

Providers can detect benchmark prompts via fingerprinting and route them to the real model, while serving substitutes for all regular traffic. Combined with randomized routing, detection becomes query-prohibitive.

5 Trusted Execution Environments are the only currently viable path to verifiable LLM APIs.

Hardware attestation cryptographically proves that the specified model weights are loaded and executed. NVIDIA H100 confidential computing makes this deployable today, with modest overhead and no reliance on statistical inference.

Call to Action: Push for TEE-backed LLM APIs as a standard offering. Until hardware attestation is standard, no commercial LLM API can be treated as transparently verifiable.